

Part II of COMP 421

Internal of a Database Management Systems

Internals of a DBMS

Query Optimization

Query Execution

Relational Operators

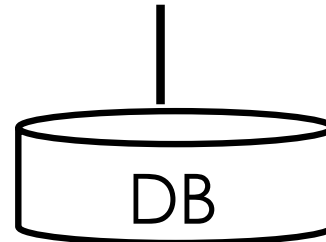
Files and Access Methods

Buffer Management

Disk Space Management

All data resides on disk

- → the disk symbol for the DB!
- I/O cost most dominant factor in execution time
- Keep number of I/Os low!



COMP 421 @ McGill

The very Essentials of Disk and Buffer Management

.

Internals of a DBMS

Query Optimization

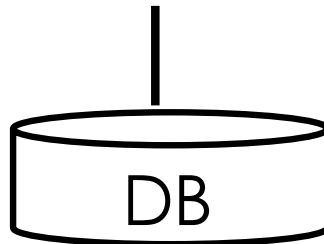
Query Execution

Relational Operators

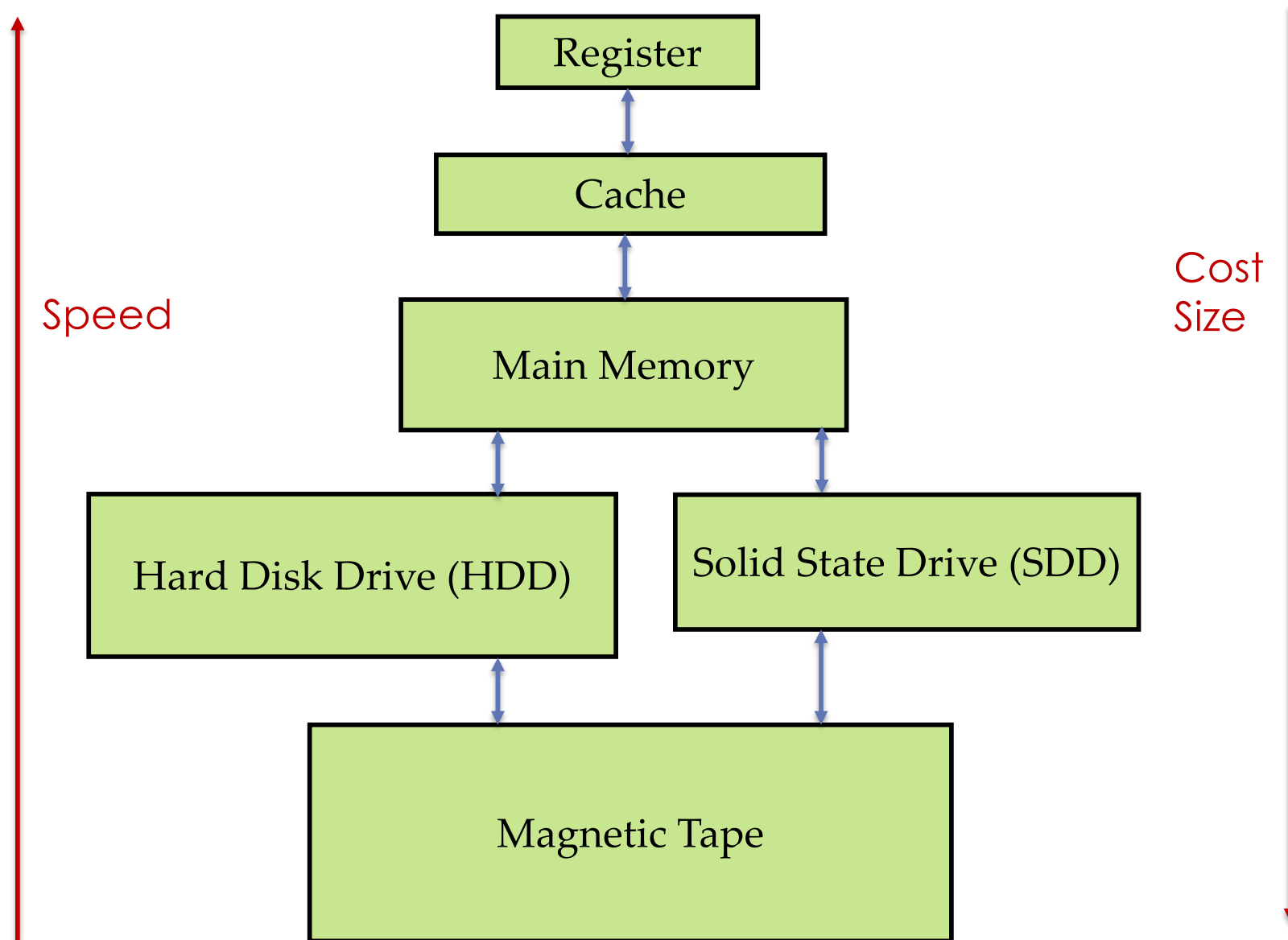
Files and Access Methods

Buffer Management

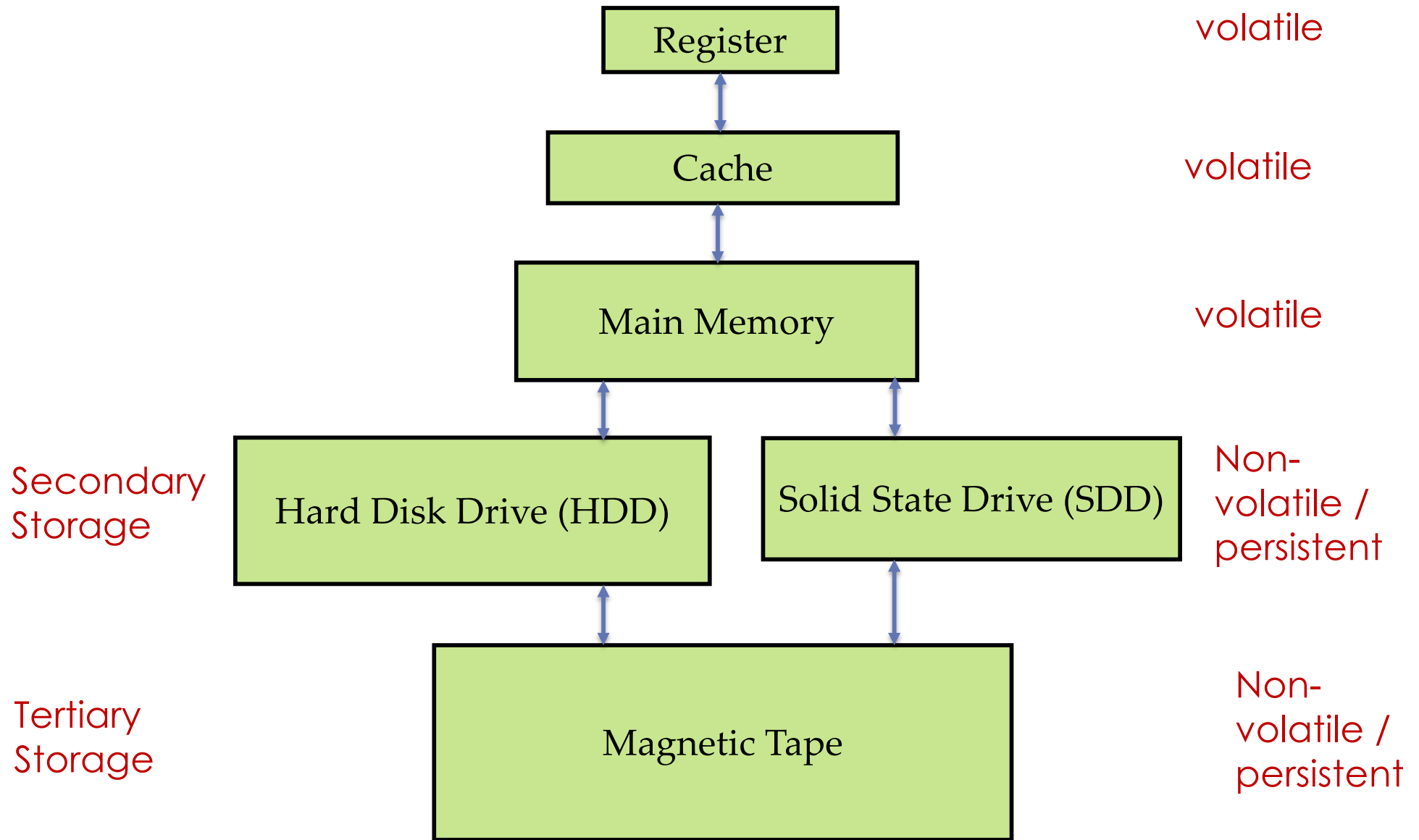
Disk Space Management



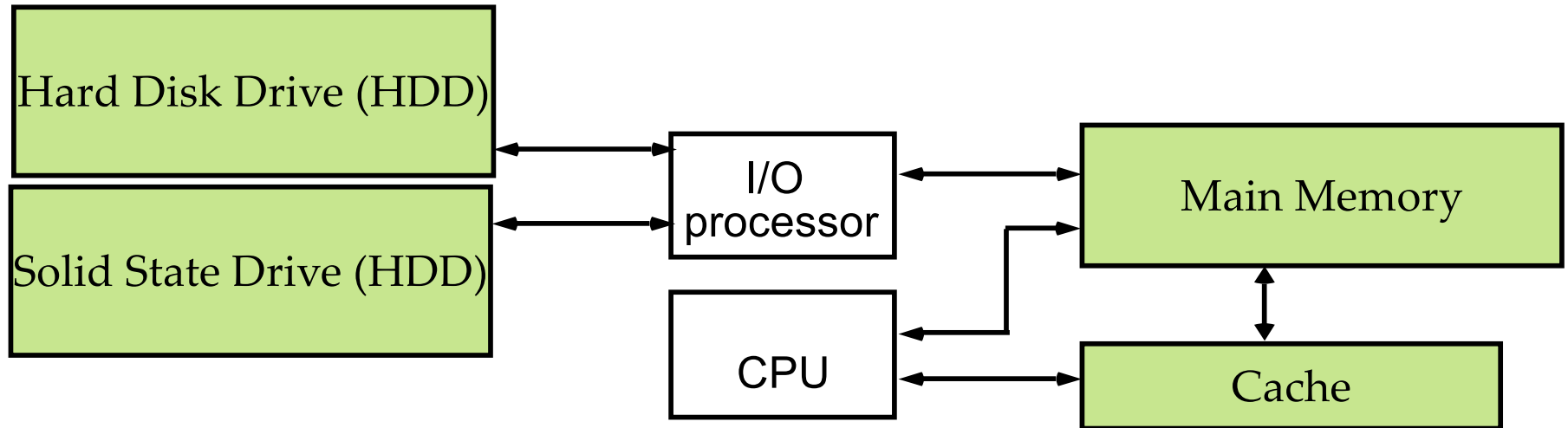
Storage Hierarchy



Storage Hierarchy



Storage Interplay



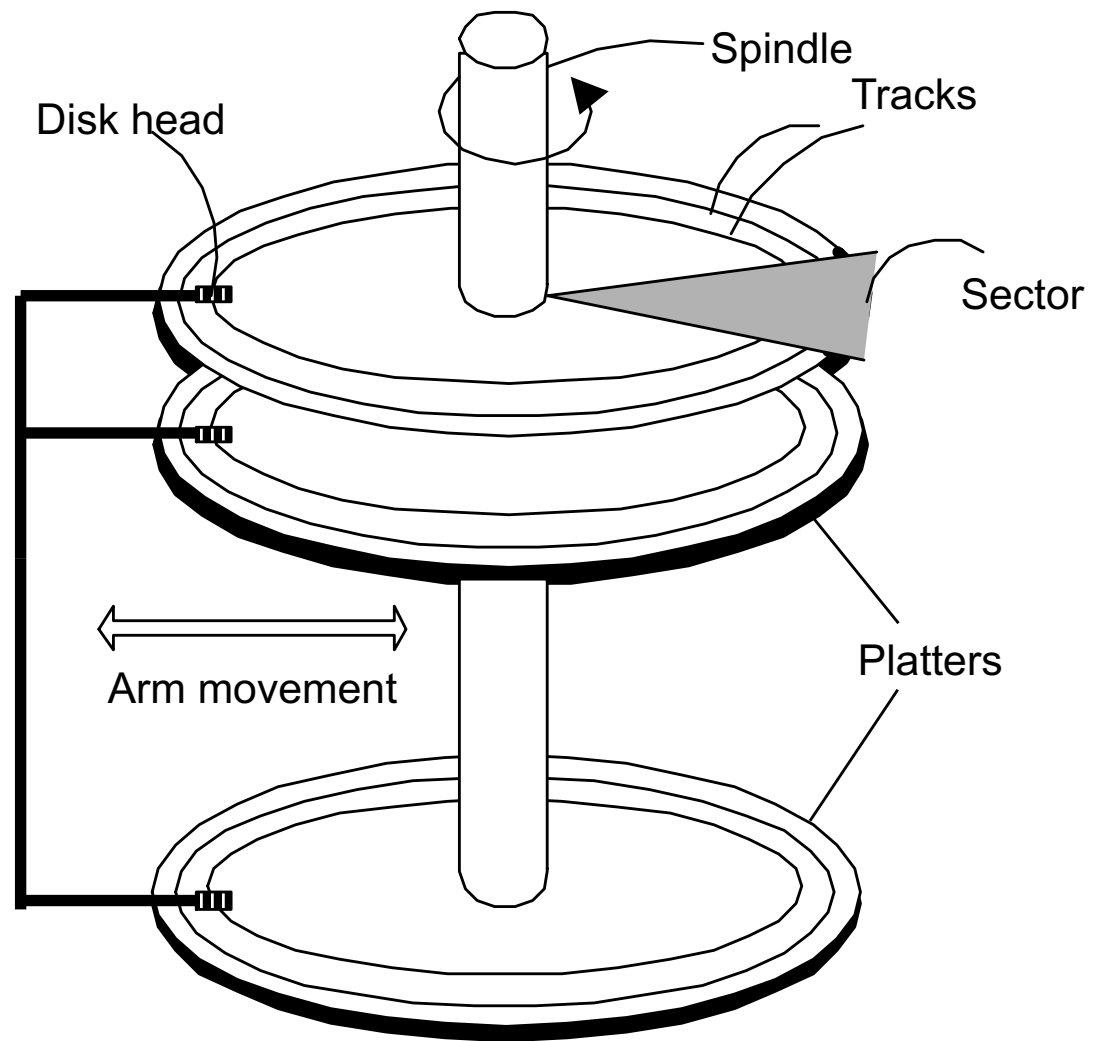
Tape drives



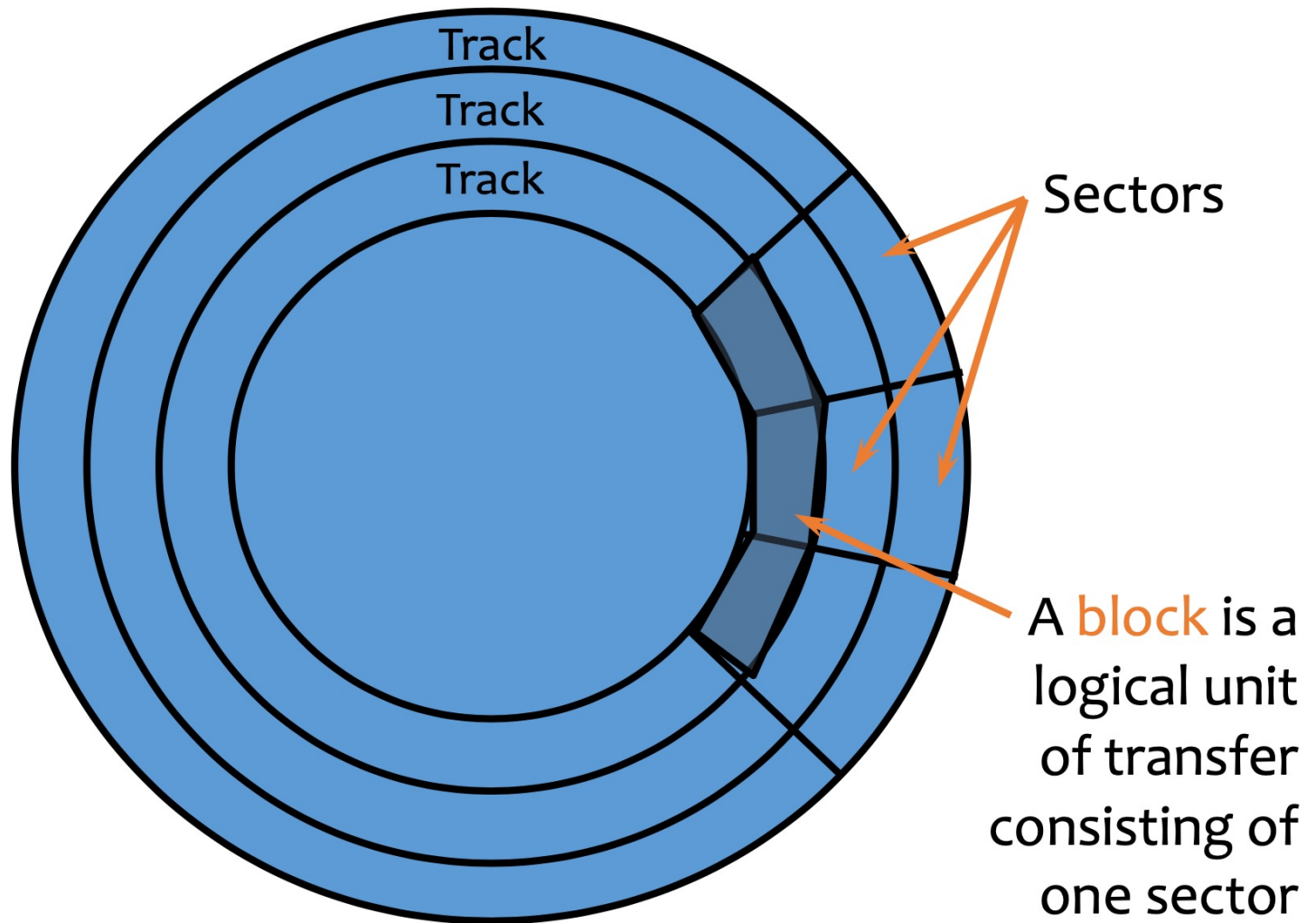
HDD







Top View



HDD

- Unit of transfer main-memory/disk: *disk blocks* or *pages*.
- Delay factors
 - Seek time: disk head moves to correct cylinder
 - Rotational delay: desired sector under the disk head
 - Transfer time for the actual block of data
- Random Disk Access
 - Delay: Seek time + rotational delay + transfer time
 - Average seek time: “typical value”: 5ms
 - Average rotational delay: “typical value: 4.2 ms
 - Transfer time: much smaller
- Sequential disk access
 - Successive requests are for successive blocks that are on the same track or on adjacent tracks
 - Seek time and Rotational delay are a one time event; only for the first block to read
 - Easily an order of magnitude faster than random access

SDD

Memory Chip instead of mechanical



No mechanical parts (flash-based)

- 1-2 orders of magnitude faster than HDD
 - Under 0.1ms vs. several ms
- Little difference between random vs. sequential read performance
- Random writes still an issue

1	Latency Comparison Numbers (~2012)					
2	-----					
3	L1 cache reference	0.5	ns			
4	Branch mispredict	5	ns			
5	L2 cache reference	7	ns			14x L1 cache
6	Mutex lock/unlock	25	ns			
7	Main memory reference	100	ns			20x L2 cache, 200x L1 cache
8	Compress 1K bytes with Zippy	3,000	ns	3	us	
9	Send 1K bytes over 1 Gbps network	10,000	ns	10	us	
10	Read 4K randomly from SSD*	150,000	ns	150	us	~1GB/sec SSD
11	Read 1 MB sequentially from memory	250,000	ns	250	us	
12	Round trip within same datacenter	500,000	ns	500	us	
13	Read 1 MB sequentially from SSD*	1,000,000	ns	1,000	us	1 ms ~1GB/sec SSD, 4X memory
14	Disk seek	10,000,000	ns	10,000	us	10 ms 20x datacenter roundtrip
15	Read 1 MB sequentially from disk	20,000,000	ns	20,000	us	20 ms 80x memory, 20X SSD
16	Send packet CA -> Netherlands -> CA	150,000,000	ns	150,000	us	150 ms

17

18 Notes

19 -----

20 1 ns = 10⁻⁹ seconds

21 1 us = 10⁻⁶ seconds = 1,000 ns

22 1 ms = 10⁻³ seconds = 1,000 us = 1,000,000 ns

23

24 Credit

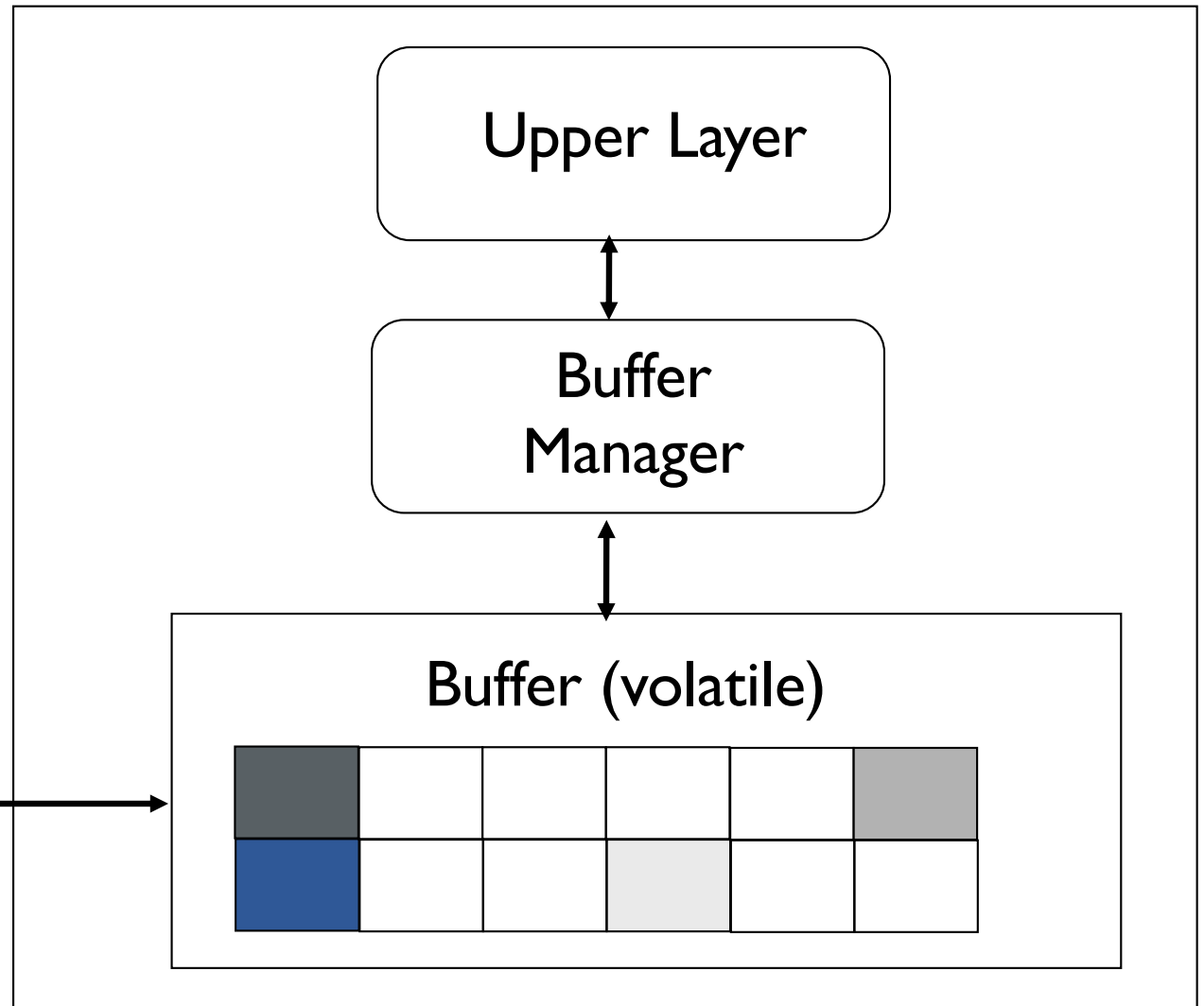
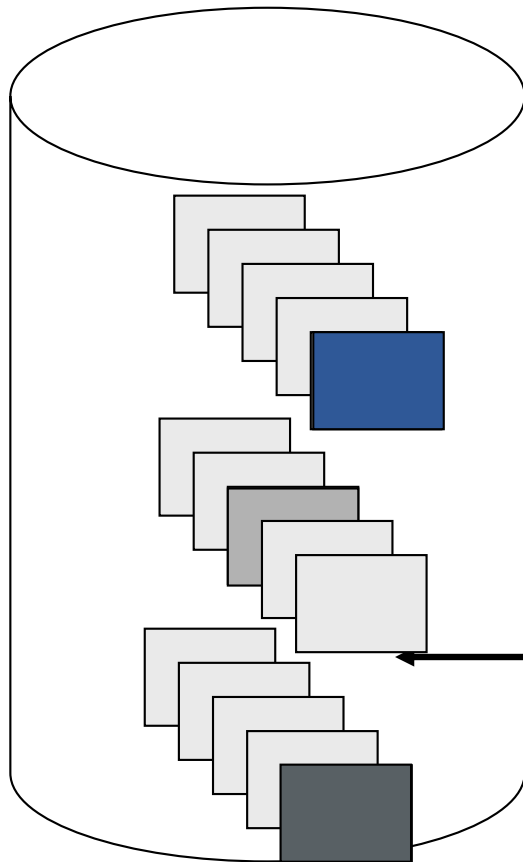
25 -----

26 By Jeff Dean: <http://research.google.com/people/jeff/>

27 Originally by Peter Norvig: <http://norvig.com/21-days.html#answers>

Architecture

Secondary
Storage
(stable)



Buffer Management in a DBMS

Page Requests from Higher Levels

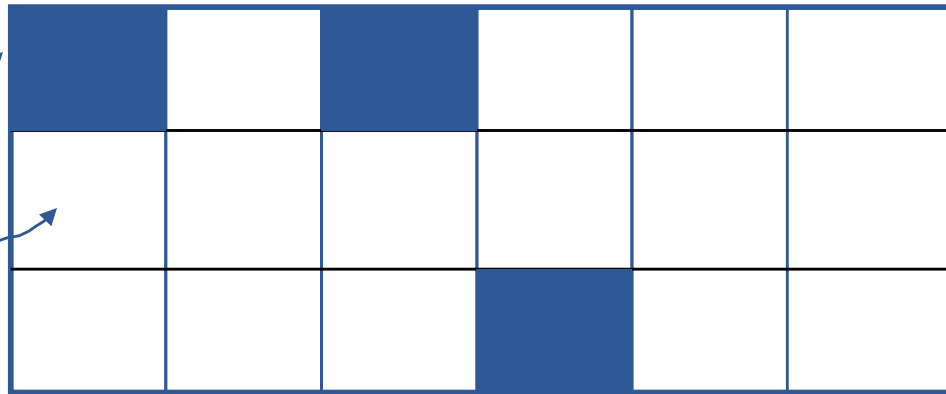
request: pageid

return: offset

BUFFER POOL

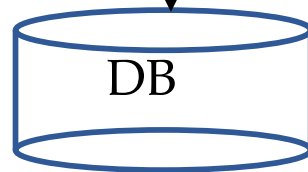
disk page

free frame



MAIN MEMORY

DISK



choice of frame dictated
by **replacement policy**

Frame no	Page id
0	202
1	Free
2	421
3	...

- ❑ Data must be in RAM for DBMS to operate on it!
- ❑ Table of <frame#, pageid> pairs is maintained.
- ❑ Some more information about each page in buffer is maintained

Loading a page from disk...

- ❑ If requested page is not in pool:
 - ☆ If there is an empty frame
 - Choose empty frame
 - ☆ Else (no empty frame)
 - Choose a frame for *replacement*
 - If current page in this frame was modified since it was read from disk (page is *dirty*)
 - Write current page back to disk (1 I/O)
 - ☆ Read requested page into chosen frame (1 I/O)

- ☆ How does the buffer manager know that upper layer is not still working on the page in the frame to be replaced?
- ☆ How does the buffer manager know whether a page is dirty?

Page Pins / Dirty Bits

Frame no	Page id	Pin counter	Dirty bit
0	202	2	F
1	431	0	T
2	...	...	

- Pin counter for each frame
- A *page request* increases pin / a *release* decreases pin
- When buffer manager returns a page (offset) to a request from upper layer (after possibly loading the page from disk):
 - Buffer manager increases pin counter of corresponding frame
- When upper layer has finished operations on page
 - Upper layer informs buffer manager (release page)
 - Buffer manager decreases pin counter of frame
- Frame is chosen for replacement by a *replacement policy*:
 - Only unpinned page can be chosen (pin count = 0)
 - Least-recently-used (LRU), Clock, MRU etc.
- Dirty bit
 - When upper layer changes page content
 - Upon release, set dirty bit
 - When page in replacement frame has dirty bit set
 - Write it back to disk before loading new page

DBMS vs. OS File System

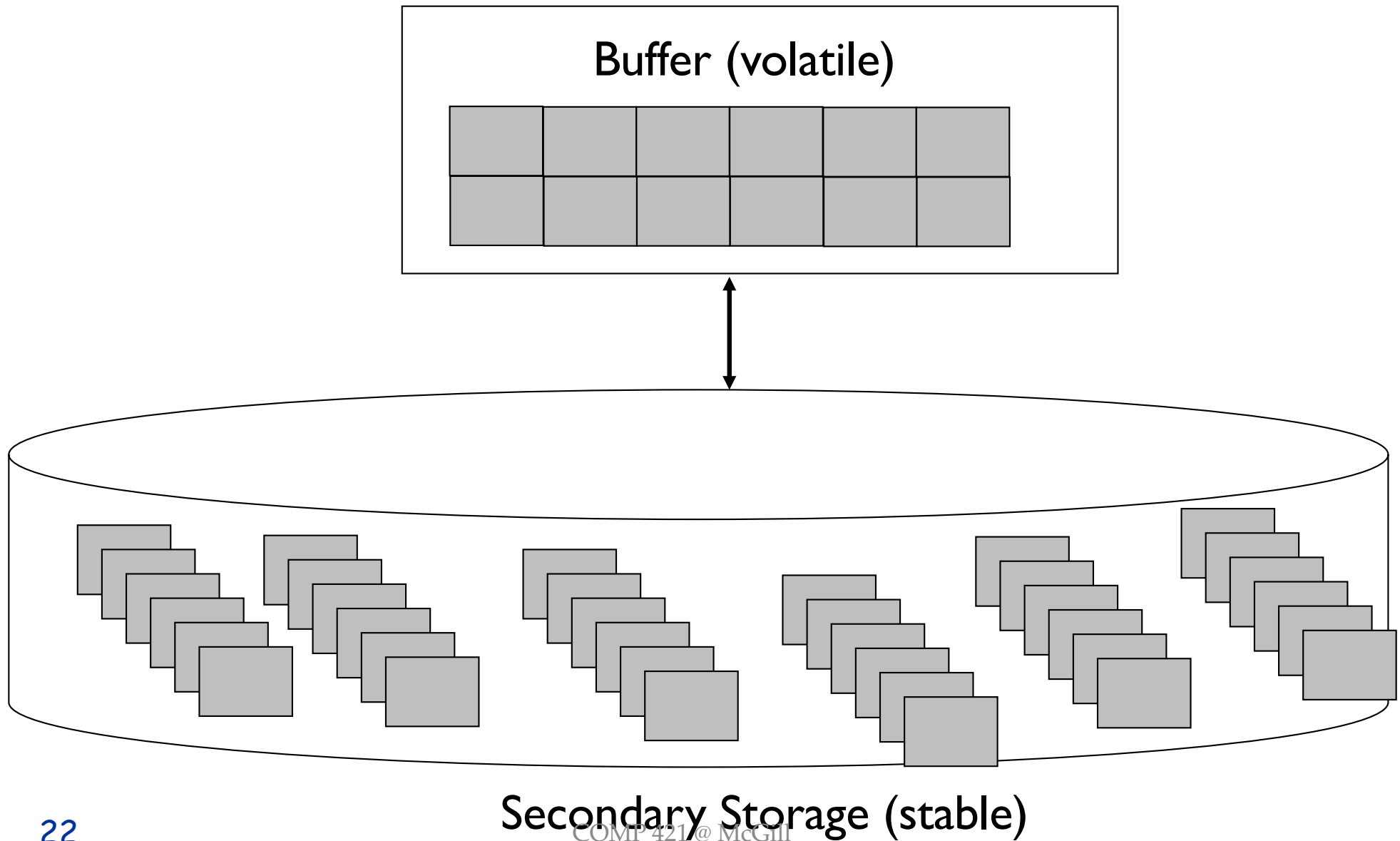
OS does disk space & buffer mgmt: why not let OS manage these tasks?

- Differences in OS support: portability issues
- Some limitations, e.g., files can't span disks.
- Buffer management in DBMS requires ability to:
 - **pin a page** in buffer pool, **force a page** to disk (important for implementing CC & recovery),
 - Optimize for DB needs
 - adjust **replacement policy**,
 - **pre-fetch pages** based on access patterns in typical DB operations.

Summary and Assumptions

- Data pages need to be in main memory to be accessed
- Page I/O is much more expensive than any computation that accesses the data in main memory
- Not all data fits into main memory
- Buffer Management to load data from disk and keep in memory as long as needed

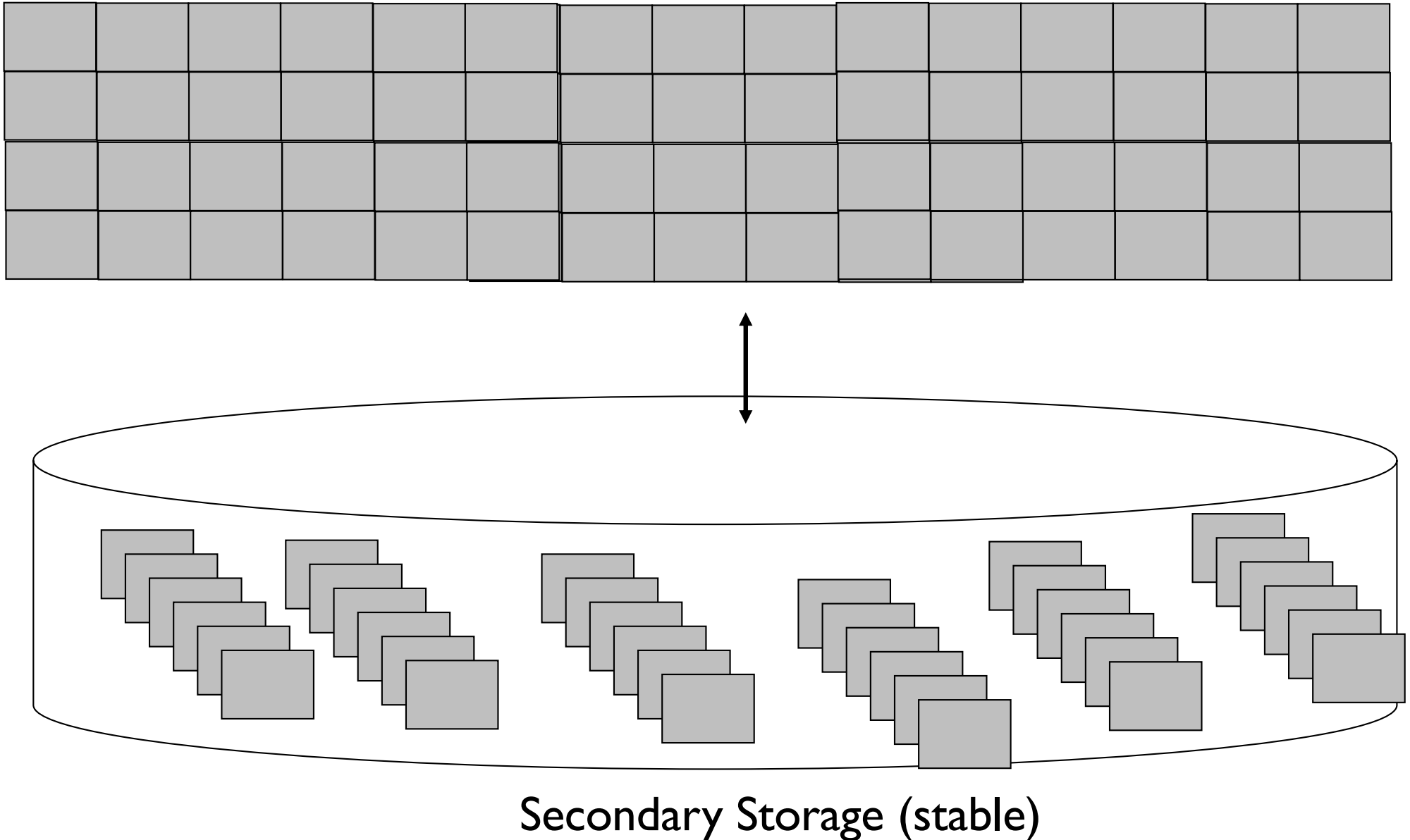
Assumed Distribution



Impact on estimating the cost of query execution

- What real database systems do:
 - Estimate
 - Number of I/Os
 - Time each I/O takes (differs depending whether sequential or random, SSD/HDD)
 - CPU time for processing data in main memory
- In this class
 - Number of I/Os
- **Back-of-the-envelope calculations!!**
 - Compare with complexity analysis, big-O, etc.

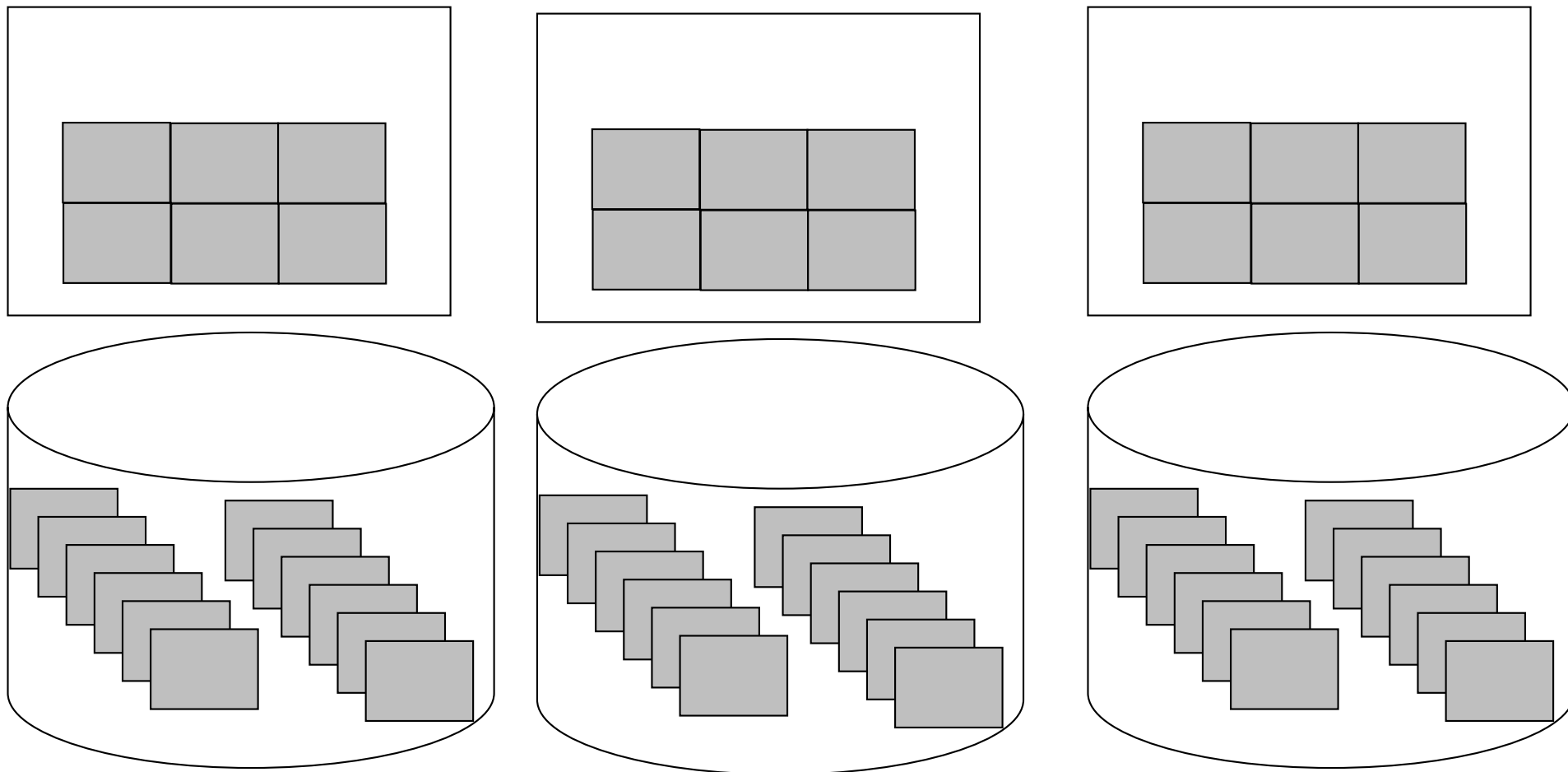
Trends we ignore: memory databases



Main-memory DBS

- Only I/O for updates (can be highly optimized)
- Optimizing for quick execution becomes main focus
- Layout in main-memory much more important
 - the concept of splitting everything into pages no more relevant
- Main memory algorithms

Later: large scale distribution



Large Scale distribution

- Communication and Coordination Overhead have huge impact on performance

Internals of a DBS I

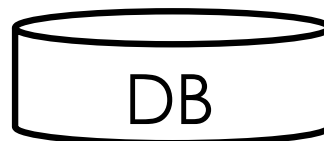
Query Optimization
And Execution

Relational Operators

Files and
Access Methods

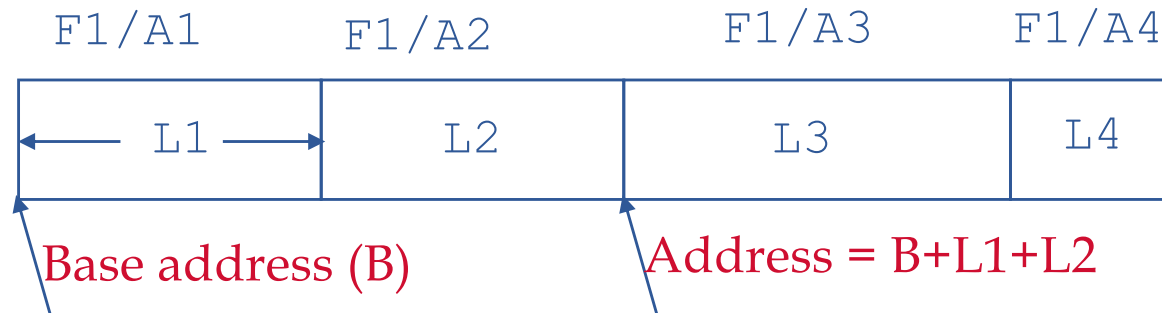
Buffer Management

Disk Space
Management



Managing Records

Record Format I: Fixed Length Attributes (Fields)

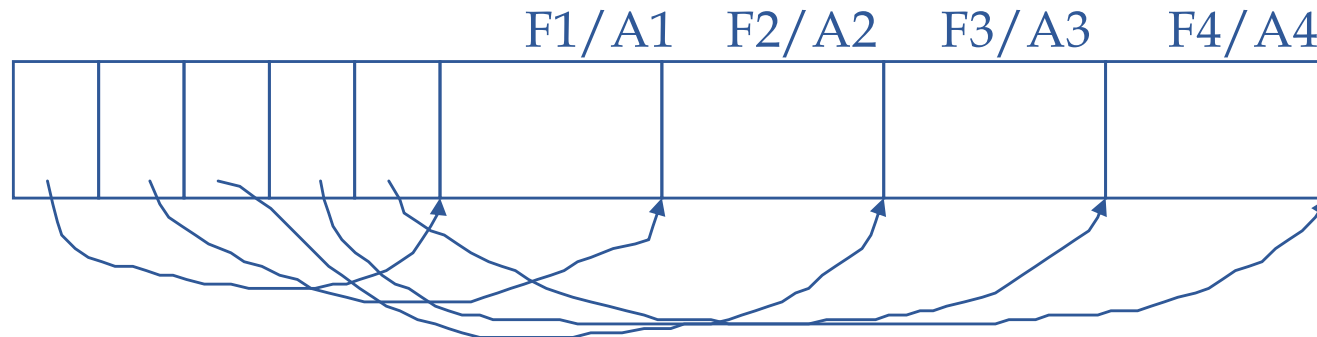


- Example: CREATE TABLE Skater (sid INT, name CHAR(20), rating float, age INT)

0	4	24	28	32
123	Lilly (padded with space)	12.5	10	

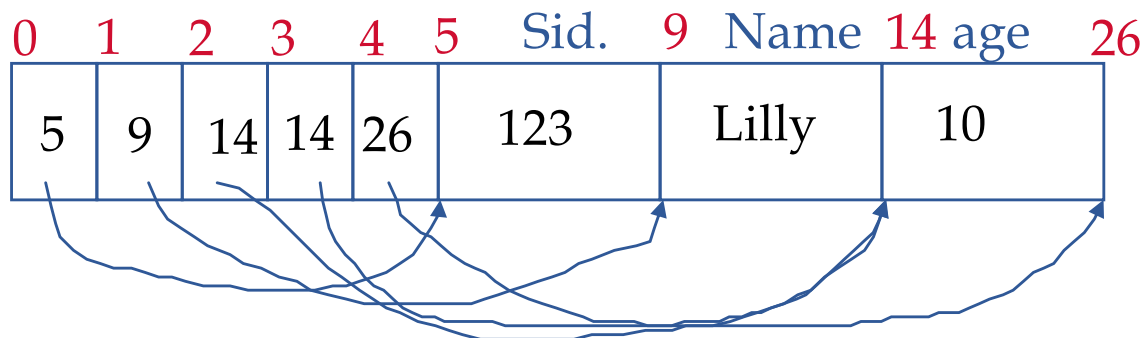
- Length of field (attribute) depends on type
- Works with fixed-length types
 - No varchar or take max. value with padding
- Offset of each field easy to calculate
- What about NULL?
 - Add a bitmap at the beginning of the record

Record Format 2: Variable Length



- ➡ Header with offset array for each attribute
- ➡ efficient storage of nulls;
- ➡ small directory overhead

- Example:
 - CREATE TABLE Skater (sid INT, name VARCHAR(20), rating float, age INT)
- Assume NULL value for rating



How to store records on a page:

Overview Visualization

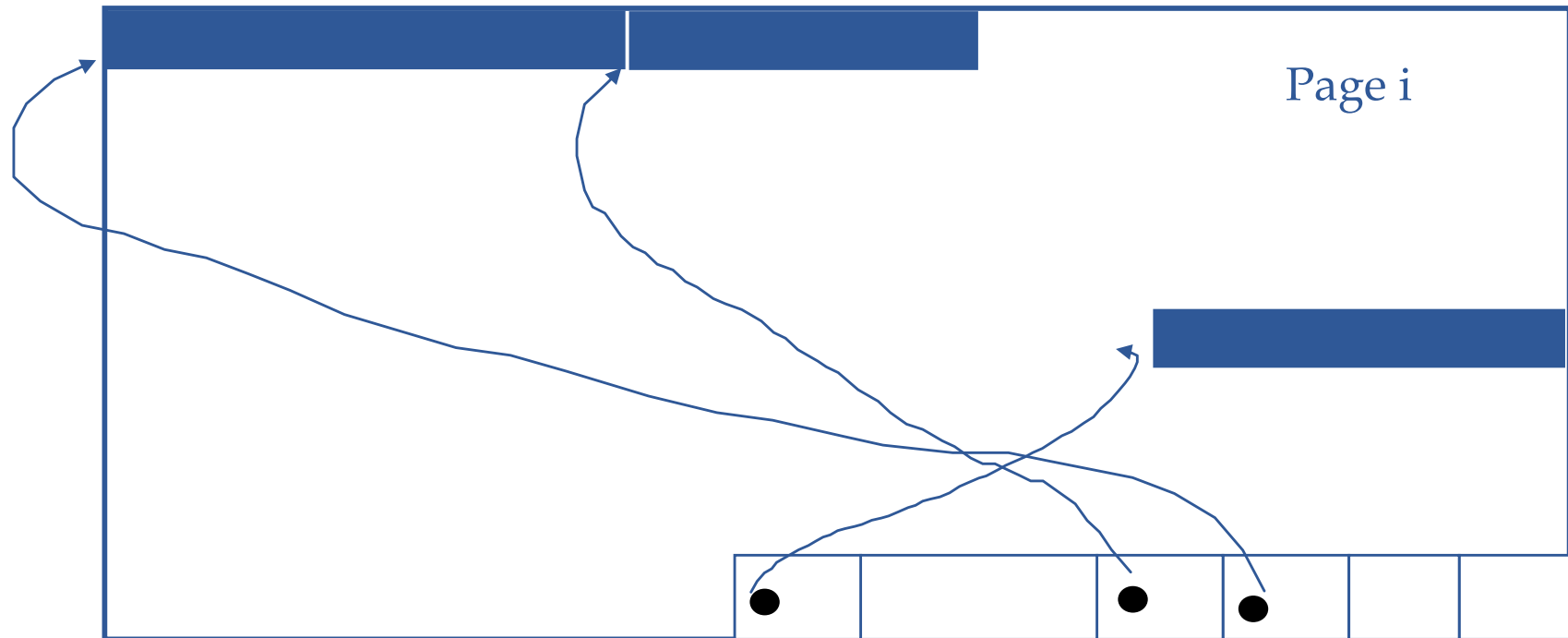
28 Yuppy 9 15	31 Debby 7 10	22 Conny 5 10	58 Lilly 10
34 Alongname 10	3 10 14	7 Averylongname 10 14	

- A page/block has typically 4KByte-8KByte
- Store Records from beginning of page
- Read like a paper page
 - From left to right, from top to bottom
- In reality: one long main memory segment of 4KB/8KB

28 Yuppy 9 15	31 Debby 7 10	22 Conny 5 10	58 Lilly 9	34 Jupiter. 2 20	...
---------------	---------------	---------------	------------	------------------	-----

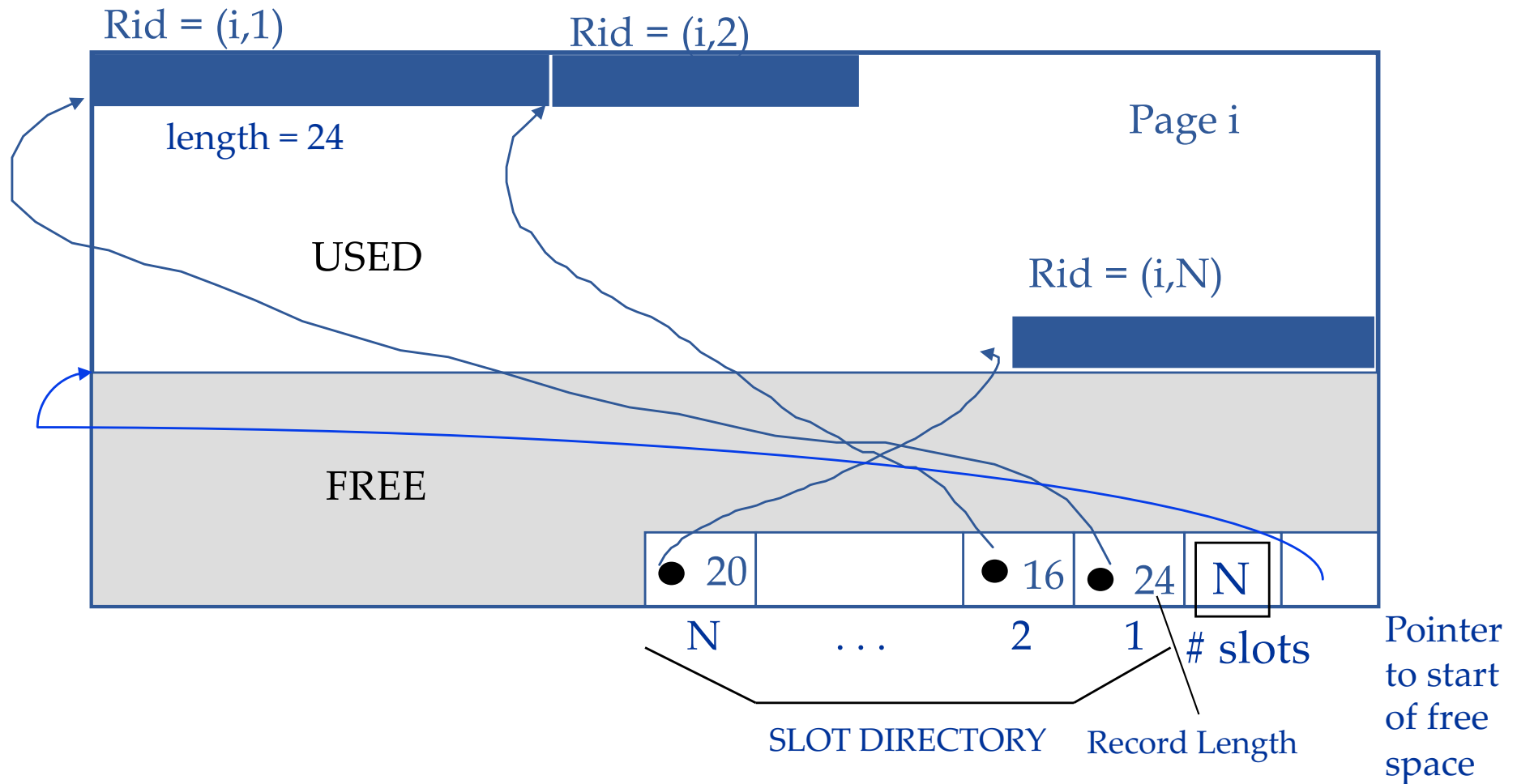
How to store records on a page:

Overview



- Store Records from beginning of page
- Use a directory at the end of each page to locate the individual records
 - Necessary for variable length records

Details

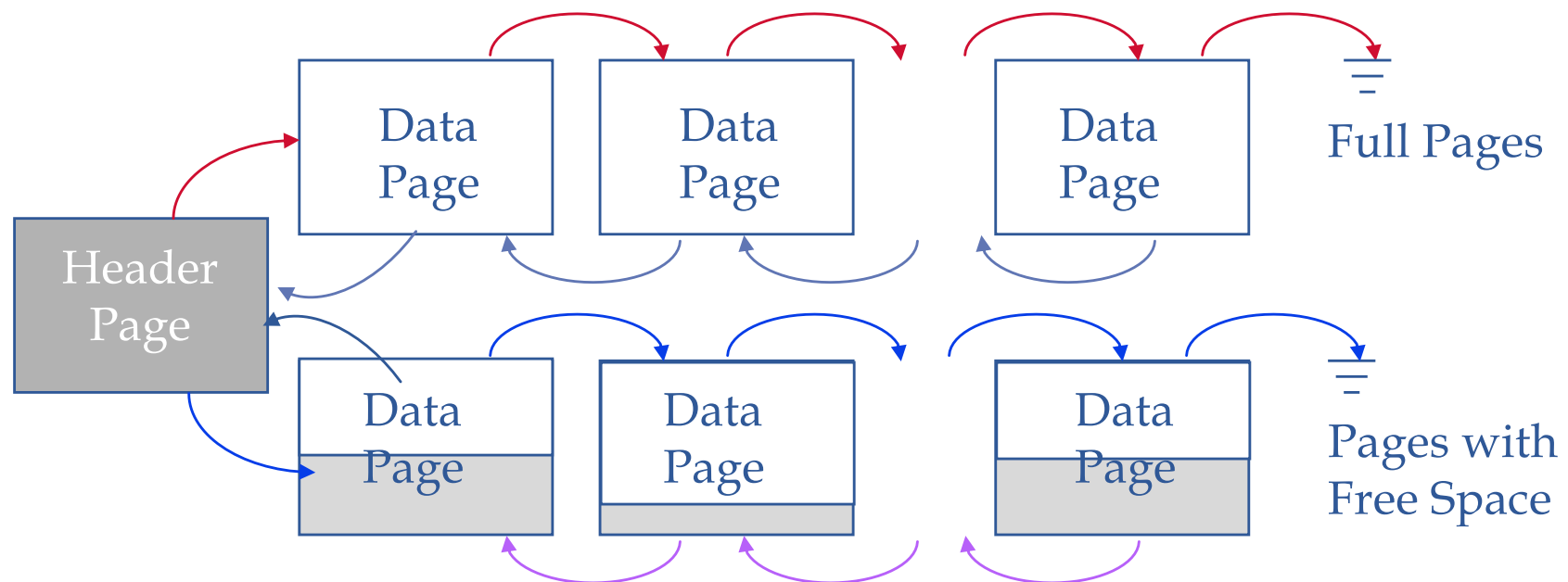


- **Record id (rid)** = internal identifier of a record: $\langle \text{page id, slot \#} \rangle$.
- After every update that changes length or after delete
 - Shift all records
 - Rids stay the same!

Managing relations in a file

- File: Collection of pages
 - For instance: One file hold records of one relation
 - All pages of file hold records of the same relation
- As file grows and shrinks, disk pages are allocated and de-allocated.
- File / Record Management:
 - keep track of the *pages* in a file
 - keep track of *free space* on pages
 - keep track of the *records* on a page
- There are many alternatives for keeping track of this.

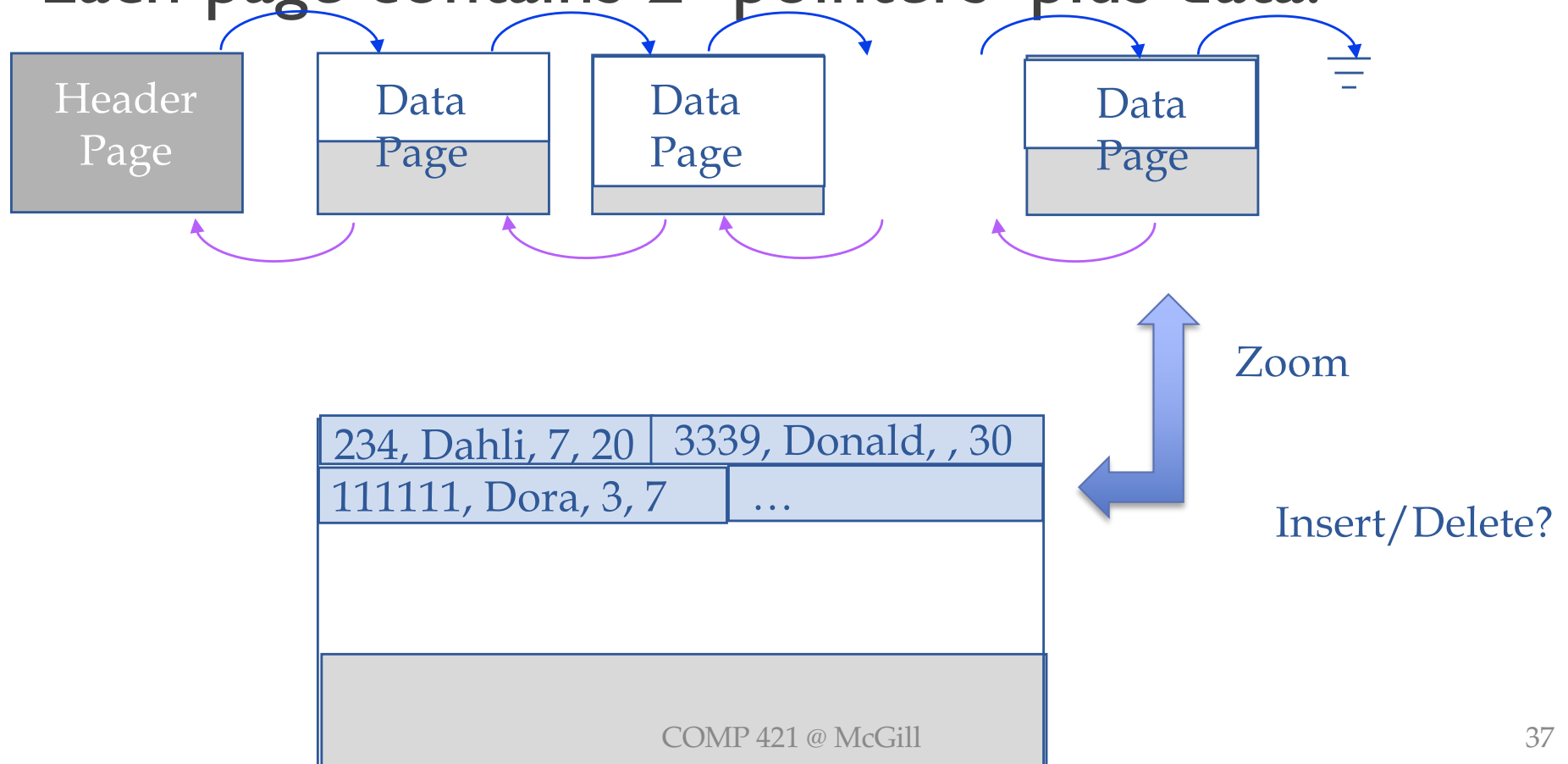
Unsorted heap file implemented as a list



- The header page id and Heap file name must be stored someplace.
- Each page contains 2 'pointers' plus data.
- Insert/delete?
- Mostly only one page with free space...

Sorted file

- Records are sorted by one of the attributes (e.g., name).
- Each page contains 2 'pointers' plus data.

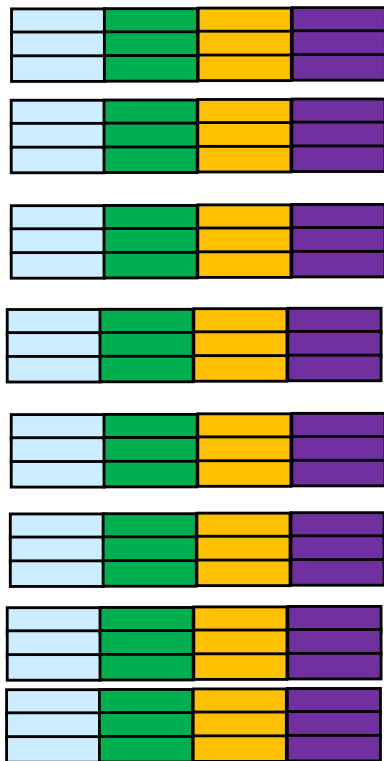


Trends we ignore: Column-based database systems

- Store each attribute/column as an array of items of the same type
 - Each page has only values of one attribute
- `SELECT sid from Skaters / SELECT avg(age) from Skaters;`
 - Row based all pages containing Skaters records must be accessed
 - Column based: only pages with sid/age must be accessed

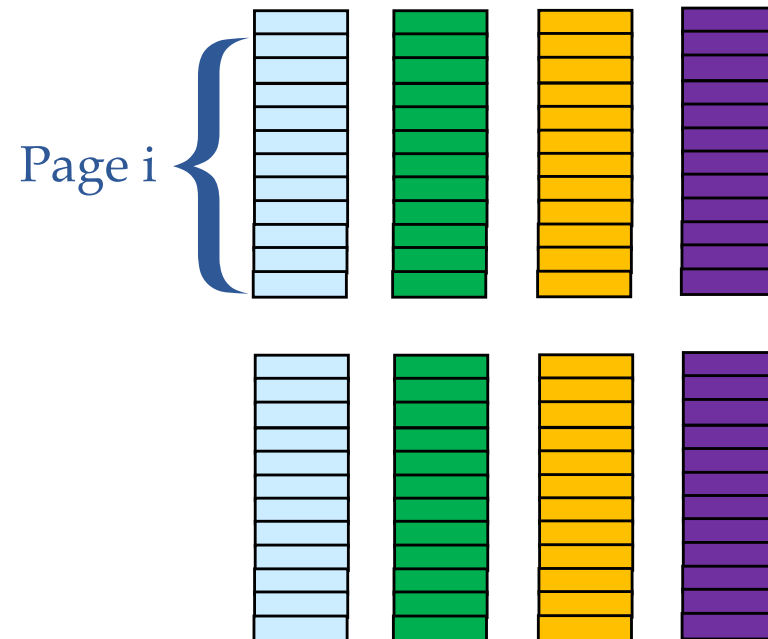
Row-based

e.g., 8 pages each with
3 record of 4 attributes



column-based

e.g., 8 pages, each page has
subset of one of the attributes



Summary Records

- Record layout:
 - fixed length vs. variable length
- Page layout for variable length records
- Relation layout
 - Store records unsorted in a list of pages
 - Store records sorted according to one of the attributes
- Many things we do not consider
 - Store by column